SNAC Hunters

Manual de la Demo de Viabilidad Técnica (D3)



Equipo 13 · Taller y Desarrollo de Proyectos Full Stack
Revisión: **jueves 15 de octubre de 2026, 12:00 h, salón MTI1**
Cliente: Pieland Games, *Attack of the SNACs*

Qué hicimos, con qué tecnologías, cómo está montado en AWS, cómo arrancarlo y cómo explicarlo.

Contenido

1. **Contexto**: qué pide la maestra y qué pide el cliente
2. **Qué hace la demo** (resumen de 1 minuto)
3. **El juego en una página**: lo mínimo de reglas para explicarlo
4. **Arquitectura**: capas, regla de oro y flujo de una partida
5. **Tecnologías**: qué usamos, para qué y dónde está
6. **Infraestructura AWS**: servicios, seguridad y despliegue
7. **Estructura del repositorio** y archivos clave
8. **Cómo arrancar el proyecto**, paso a paso
9. **Guion de la demo** (10 minutos)
10. **Calidad**: pruebas, cobertura y CI
11. **Decisiones de diseño** registradas
12. **Preguntas probables** con respuesta
13. **Glosario**
14. **Estado actual y siguientes pasos**

> **Cómo usar este manual:** en la sesión de preguntas la maestra puede preguntarle a **cualquier integrante** sobre cualquier documento (D1 a D4). Todos deben leer al menos las secciones 2, 4, 5, 6 y 12. Lo más importante para el jueves es la sección 9 (guion).

1. Contexto

La revisión del jueves (correo de la profesora)

Bloque	Tiempo	Qué hay que hacer
Cronograma en Notion (D4)	~10 min	Demo en vivo del tablero: estados, tareas por integrante, sprints y fechas. Detallar las actividades del **Sprint 1** de cada integrante. (Lo prepara el equipo; no está en este manual.)
Demo de Viabilidad Técnica (D3)	~10 min	Demo funcional que valide el **stack propuesto en D3**. **Es lo que cubre este manual.**
Preguntas individuales	~10 min	Preguntas directas a cualquier integrante sobre D1, D2, D3 o D4.

Lo que pide el cliente (Pieland Games)

- **Brief original:** vertical slice web para **un jugador** en la zona **Sherbet Forest**. Un jugador nuevo debe entender los dados en **menos de 5 minutos**. Además: persistencia, analítica, mobile-first, validación en servidor y economía configurable.
- **Reunión con el cliente:** lo que más le interesa es una **guía interactiva / tutorial**, porque el juego de mesa tiene muchas reglas. **No** quiere multijugador. El backend debe ser **solo con servicios de AWS**.

Por eso la demo es un **tutorial interactivo jugable** que recorre todo el stack: navegador, API, base de datos e infraestructura en AWS.

2. Qué hace la demo

Una app web (React) donde el **Huntmaster**, el personaje del juego, enseña paso a paso a cazar un SNAC. Después el jugador puede cazar libremente. Las reglas viven en un motor compartido y el servidor valida cada partida.

🎓 Tutorial guiado (8 pasos)

Contra *Garden Salaad*. Explica espacios, dados, comodín, relanzar, dado blanco de daño, corazones como escudo y cómo terminar la caza. Los textos vienen de `config/tutorial.json`.

🎲 Combate real con dados

Al tocar un dado **se iluminan solo los espacios donde sirve**. El botón principal muestra en vivo si el resultado sería Huir, Estudiar o Derrotar.

🎄 Caza libre

El jugador elige entre **2 SNACs de tipos distintos** (estructura de run del brief). Están los 9 monstruos reales de Sherbert Forest.

✅ Validación en servidor

Al terminar, la API vuelve a jugar la partida con el mismo motor y responde "*✓ Validado por el servidor*". Las trampas se rechazan.

💾 Persistencia

El progreso (puntos, trofeos, KOs) y el combate en curso se guardan en el navegador (IndexedDB). Si recargas, vuelves al mismo punto.

📊 Analítica + botón DEV

Registra sesiones, monstruos vistos, tiradas y resultados. El botón **DEV** (solo en desarrollo) muestra el resumen local y el del servidor.

🔀 Feature toggles

En `config/game.json` se activan o desactivan funciones (tutorial, caza libre, validación, pueblo, jefe) **sin cambiar código**.

📱 Mobile-first

Botones de al menos 48 px y acciones principales fijas abajo (zona del pulgar). Se abre desde el celular en la misma red Wi-Fi.

> **En una frase:** "Construimos el tutorial que pidió el cliente usando todo nuestro stack: React en el cliente, un motor de reglas en TypeScript compartido, una API que en AWS corre en Lambda y valida las partidas re-simulándolas, DynamoDB para datos y toda la infraestructura definida como código con CDK."

3. El juego en una página

Attack of the SNACs es un juego de **"arriesga tu suerte"** (push your luck): lanzas dados para igualar los símbolos de un monstruo. Puedes relanzar cuantas veces quieras, pero cada relanzamiento agrega un dado de daño.

Símbolos y colores

Código	Ícono en el arte	Color (dado especial / trofeo)
`Horn`	Axe (hacha)	Naranja
`Wing`	Arrow (flecha)	Azul
`Claw`	Finder (lupa)	Verde
`Tail`	Hook (gancho)	Morado

Los dados

Dado	Caras	Para qué sirve
Rojo (estándar)	Hacha, flecha, lupa, gancho + **2 corazones**	Base del jugador. El corazón es un **escudo** contra el daño.
Especial (de color)	4 veces su símbolo + **2 comodines**	El comodín cuenta como **cualquier símbolo**, nunca como corazón.
Blanco (daño)	**3 corazones rotos**, 2 en blanco, 1 comodín	Se agrega **cada vez que relanzas**. El corazón roto quita 1 HPie.

Fuente: `Dice-guide-V03.png` del cliente. Los valores están en `config/dice.json`.

Cómo termina una caza

Resultado	Condición	Consecuencia
Derrotar	Los 4 espacios cubiertos	+1 punto y eliges un trofeo
Estudiar	2 o 3 espacios cubiertos	Eliges un trofeo; el monstruo escapa
Huir	1 espacio cubierto	Sin premio, pero no recibes más daño
KO	El HPie (vida, máximo 6) llega a 0	−1 punto. Si en esa misma tirada completaste los 4 espacios, conservas la derrota.

- Un espacio con estrella acepta **cualquier símbolo**.
- Un espacio con corazón roto solo se cubre **recibiendo daño** (Porcupineapple, Picmimic, Porkchop).
- **Daño y escudo:** cada corazón roto cuesta 1 HPie, salvo que haya un corazón en tu reserva; ese corazón se gasta y bloquea el daño.

Los 9 SNACs de Sherbert Forest

Monstruo	Tipo	Espacios	Trofeos
Garden Salaad	Claw	Claw, Claw, Horn, cualquiera	Claw / Horn
Guac-a-mole	Claw	Claw, Claw, Tail, Wing	Claw / Tail
Porcupineapple	Claw	Claw, Claw, Horn, **daño**	Claw / Horn
Broccoltree	Horn	Horn, Horn, Wing, cualquiera	Horn / Wing
Picmimic	Horn	Horn, Horn, Claw, **daño**	Horn / Claw
Spinachasaurus	Tail	Tail, Tail, Wing, cualquiera	Tail / Wing
Porkchop	Tail	Tail, Tail, Claw, **daño**	Tail / Claw
Beeholder	Wing	Wing, Wing, Horn, cualquiera	Wing / Horn
Chickagriff	Wing	Wing, Wing, Tail, cualquiera	Wing / Tail

Fuente: `Sherbert Forest monster list.xlsx` del cliente → `config/monsters.json`. El jefe **Pieous** aún no está implementado.

4. Arquitectura

Es un **monorepo** (un solo repositorio con varios paquetes) manejado con **pnpm workspaces**. Cada capa tiene una sola responsabilidad y las dependencias van **de arriba hacia abajo**.

`apps/web`: interfaz React. Solo dibuja el estado y manda acciones. Nunca calcula reglas.

`apps/api`: API serverless (Lambda). Valida partidas re-simulándolas y guarda datos. Nunca confía en el cliente.

▼ ambos importan el mismo motor ▼

`packages/engine`: todas las reglas del juego. TypeScript puro, **sin** React, AWS ni archivos (sin I/O). Incluye el generador de números aleatorios con semilla.

▼ valida con Zod ▼

`config/`: JSON con los datos del juego: monstruos, dados, puntos, vida, textos del tutorial y feature toggles.

`infra/`: la infraestructura de AWS escrita como código con **AWS CDK** (TypeScript).

Las "reglas de oro" (lo que más vale saber explicar)

1. **La UI nunca calcula reglas.** Todo pasa por el engine. Ejemplo: para iluminar los espacios, la UI le pregunta al engine con `validSlotsFor()`.
2. **El servidor re-simula semilla + acciones** con el mismo engine. El cliente nunca le dice al servidor "gané"; le dice "esto fue lo que hice".
3. **Nada de valores fijos en el código.** Puntos, vida, dados, monstruos y textos están en `config/` (el cliente ajustará la economía después).
4. **El engine es puro.** ESLint prohíbe importar `react`, `@aws-sdk`, `fs` o `path` dentro de `packages/engine`.
5. **TypeScript estricto** en todos los paquetes.
6. **Cobertura de pruebas ≥ 90 %** en el engine; el CI falla si baja.

¿Por qué funciona la validación? Determinismo

Los dados no usan `Math.random()`. Usan **Mulberry32**, un generador con **semilla** (*seed*): con la misma semilla y las mismas acciones el resultado es **siempre idéntico**. Así el servidor puede repetir la partida y comprobar que el resultado es legítimo.

Flujo de una partida, de principio a fin

1. El jugador inicia una caza. El navegador genera una **semilla** y el engine lanza los dados con `startCombat()`.

2. Cada toque (asignar, quitar, relanzar, resolver) es una **acción**. El engine devuelve el nuevo estado con `applyAction()`; si la acción es ilegal, la rechaza.
3. Zustand guarda en IndexedDB **solo la semilla y la lista de acciones**, no el estado calculado.
4. Si se recarga la página, el estado se reconstruye con `replayCombat()`. Si los datos guardados están corruptos, se descartan sin perder el progreso.
5. Al terminar, el navegador hace `POST /runs` con `{ monsterId, seed, actions }`.
6. La Lambda valida el formato con **Zod**, re-simula con el **mismo engine** usando los dados y la vida de `config/` (no los del cliente) y calcula puntos y trofeo.
7. Si es válido, guarda la partida en **DynamoDB** y responde `201 { valid: true }`. Si hubo trampa o una acción ilegal, responde `422`.
8. En paralelo, los eventos de analítica se envían en lotes a `POST /events`.

5. Tecnologías

Base del proyecto

Tecnología	Para qué la usamos	Dónde está
TypeScript (strict)	Un solo lenguaje en front, back, motor e infraestructura. Los tipos se comparten (por ejemplo, `Monster` y `CombatAction`).	Todo el repo; `tsconfig.base.json`
Node.js	Entorno para ejecutar herramientas y la API local. Desarrollo en 20+; Lambda en 22.	`.nvmrc`
pnpm workspaces	Gestor de paquetes y del monorepo. `--filter` ejecuta comandos en un paquete específico.	`pnpm-workspace.yaml`
Zod	Valida datos en tiempo de ejecución: los JSON de `config/` y todo lo que llega a la API.	`engine/src/schemas.ts`, `api/src/app.ts`
ESLint + **Prettier**	Calidad y formato. ESLint además **impide** que el engine importe React, AWS o archivos.	`eslint.config.js`, `.prettierrc`

Frontend (apps/web)

Tecnología	Para qué	Ejemplo en la demo
React 18	Componentes de interfaz.	`screens/CombatScreen.tsx`
Vite	Servidor de desarrollo con recarga instantánea y build de producción.	`pnpm ... web dev`
Tailwind CSS	Estilos con clases utilitarias; facilita el diseño responsivo mobile-first.	Colores de dados en `tailwind.config.js`
Zustand	Estado global (pantalla, combate, progreso). Su middleware `persist` lo guarda automáticamente.	`game/store.ts`
Framer Motion	Animaciones: dados que ruedan, espacios que laten, globo del Huntmaster.	`components/DieView.tsx`
idb-keyval	Guarda datos en **IndexedDB**, la base de datos del navegador que sobrevive al cierre.	`game/persistence.ts`
AWS Amplify	Cliente de autenticación con Cognito (obtiene el token para la API). Conectado, aún sin pantalla de login.	`game/auth.ts`

Backend (apps/api)

Tecnología	Para qué	Archivo
AWS Lambda	Ejecuta la API sin administrar servidores; se cobra por uso.	`src/lambda.ts`

Router independiente	La lógica HTTP no depende de AWS. La usan la Lambda, el servidor local y las pruebas.	`src/app.ts`
AWS SDK v3 (DynamoDB)	Leer y escribir partidas y eventos.	`src/dynamo-store.ts`
Servidor local (node:http + tsx)	Corre el mismo router en tu PC, con almacenamiento en memoria, para desarrollar sin AWS.	`src/local.ts`
esbuild	Empaqueta la Lambda en un solo archivo (CDK lo usa automáticamente).	`package.json`

Pruebas y CI

Tecnología	Para qué
Vitest	Framework de pruebas en todos los paquetes; incluye medición de cobertura (v8).
fast-check	*Property-based testing*: genera cientos de entradas aleatorias para comprobar propiedades (por ejemplo, "misma semilla, mismo resultado").
jsdom	Simula el navegador para probar la persistencia de la web.
GitHub Actions	CI en cada push: `install → lint → typecheck → test`.

6. Infraestructura AWS

Todo está definido **como código** en `infra/lib/snac-hunters-stack.ts` con **AWS CDK**. CDK traduce TypeScript a una plantilla de **CloudFormation** y AWS crea los recursos. Nada se configura a mano en la consola. Región: **us-east-1**. Ambientes: **dev** y **prod**.

CloudFront + S3
Hosting de la app React. S3 privado; solo CloudFront puede leerlo (OAC). HTTPS obligatorio.

Cognito
User Pool: registro e inicio de sesión de jugadores con email. Emite tokens JWT.

API Gateway
API REST con **Cognito authorizer**, CORS y límite de peticiones (throttling).

Lambda
Node.js 22. Ejecuta el engine para re-simular partidas. Logs en CloudWatch (30 días).

DynamoDB
Base de datos NoSQL, pago por uso.

Diseño single-table
`pk = RUN#<usuario>` · `sk = fecha#runId` para las partidas validadas.
`pk = EVENT#<sesión>` · `sk = timestamp#n` para los eventos de analítica.

Camino de una petición

```
Navegador ──HTTPS──▶ CloudFront ──▶ S3              (descarga la app React)
Navegador ──HTTPS + token JWT──▶ API Gateway
                                 │ 1. Cognito authorizer valida el token
                                 ▼
                              Lambda (engine) ──▶ DynamoDB
```

Endpoints

Ruta	Auth	Qué hace
`GET /health`	No	Comprueba que la API está viva y cuántos monstruos cargó.
`POST /runs`	Cognito	Recibe semilla + acciones, re-simula, calcula resultado y puntos, y guarda en DynamoDB.
`POST /events`	Cognito	Recibe lotes de eventos de analítica (máximo 100 por lote).
`GET /stats`	Cognito	Resumen de analítica (lo muestra el botón DEV).

Seguridad (por qué el cliente no puede hacer trampa)

- El cliente **no manda resultados**, solo acciones. Si manda `"points": 999`, se ignora; hay una prueba que lo verifica.
- Los dados iniciales y la vida **los decide el servidor** desde `config/game.json`.
- Cualquier acción ilegal (dado inexistente, símbolo que no coincide, dado usado dos veces) → `422`.
- Toda entrada se valida con Zod antes de procesarse (límites de tamaño incluidos).
- S3 no es público; API Gateway limita las peticiones por segundo.

Por qué serverless

- **Costo:** se paga por uso (Lambda, DynamoDB on-demand); con poco tráfico es prácticamente gratis.
- **Escala** sola, sin administrar servidores.
- Lo pidió el cliente: **solo servicios de AWS**.

Comandos de infraestructura

```
# Genera la plantilla de CloudFormation (NO requiere cuenta de AWS). Sirve para la demo.
pnpm --filter @snac-hunters/infra synth

# Desplegar (requiere AWS CLI configurado con credenciales: `aws configure`)
pnpm --filter @snac-hunters/web build                 # genera apps/web/dist
pnpm --filter @snac-hunters/infra run deploy -c stage=dev  # "run" es obligatorio: deploy es comando de
pnpm
```

> **Importante:** hoy la infraestructura **sintetiza sin errores pero no está desplegada** (no tenemos cuenta de AWS configurada en la laptop). Si la maestra pregunta, dilo con honestidad: el código de infraestructura está listo y validado con `cdk synth`; el despliegue es el siguiente paso.

7. Estructura del repositorio

```
SNAC-Hunters/
├── packages/engine/src/
│   ├── types.ts            tipos: GameSymbol, Slot, Die, Monster…
│   ├── schemas.ts          schemas Zod de config/
│   ├── loader.ts           loadMonsters, loadDiceTable, loadGameConfig, loadTutorial
│   ├── rng.ts              SeededRng (Mulberry32)
│   ├── combat.ts           ★ motor de combate: startCombat, applyAction, replayCombat
│   └── __tests__/          69 pruebas (unitarias + property-based)
├── apps/api/src/
│   ├── app.ts              ★ router HTTP: /health /runs /events /stats
│   ├── lambda.ts           adaptador para AWS Lambda
│   ├── local.ts            servidor local en :3001
│   ├── store.ts / dynamo-store.ts  persistencia (memoria / DynamoDB)
│   └── __tests__/          9 pruebas de integración
├── apps/web/
│   ├── public/assets/      arte del cliente: monstruos, íconos, trofeos, fuente
│   └── src/
│       ├── game/store.ts   ★ estado Zustand + persistencia + conexión con el engine
│       ├── game/analytics.ts, api.ts, auth.ts, config.ts, persistence.ts
│       ├── screens/        Home, Pick, Combat, Result
│       └── components/     DieView, MonsterCard, Coach, HpBar, DevPanel
├── config/
│   ├── monsters.json  dice.json  game.json  tutorial.json
├── infra/
│   ├── bin/snac-hunters.ts       entrada de CDK (elige stage)
│   └── lib/snac-hunters-stack.ts ★ todos los recursos de AWS
├── docs/decisiones.md  docs/demo-d3.md
└── .github/workflows/ci.yml
```

★ = los archivos que conviene tener abiertos durante la demo.

Archivos de configuración (`config/`)

Archivo	Contenido
`monsters.json`	Los 9 SNACs: tipo, 4 espacios, 2 trofeos y su frase.
`dice.json`	Las 6 caras de cada tipo de dado.
`game.json`	Vida máxima (6), dados iniciales (4 rojos + 1 verde), monstruo y semilla del tutorial, **feature toggles** y **puntuación** (derrota +1, KO −1).
`tutorial.json`	Los 8 pasos del Huntmaster: título, texto, qué resaltar y qué acción avanza el paso.

8. Cómo arrancar el proyecto

Requisitos (una sola vez)

- **Node.js 20 o más reciente**: comprobar con `node -v`
- **pnpm 10**: instalar con `npm install -g pnpm@10`; comprobar con `pnpm -v`
- **Git**, y el repositorio clonado en la rama de la demo: `feat/demo-viabilidad-d3`

Paso 1: Entrar a la carpeta e instalar dependencias

```
cd "C:\...\SNAC-Hunters"
pnpm install          # descarga las librerías de TODOS los paquetes del monorepo
```

Paso 2: Terminal 1, la API

```
pnpm --filter @snac-hunters/api dev
```

Debe aparecer: `API local (mismo handler que Lambda) en http://localhost:3001`. **No cierres esta terminal.**

`--filter` ejecuta el comando solo en ese paquete. `dev` corre `tsx watch src/local.ts`, que se reinicia solo si cambias código.

Paso 3: Terminal 2, la web

```
pnpm --filter @snac-hunters/web dev
```

```
  VITE v5.4.21  ready in 500 ms
  ➜  Local:   http://localhost:5173/       ← en la laptop
  ➜  Network: http://192.168.x.x:5173/     ← en el celular (misma red Wi-Fi)
```

Paso 4: Abrir y comprobar

- Juego: **http://localhost:5173**. La primera vez puede tardar o quedarse en blanco unos segundos mientras Vite prepara las librerías; recarga si pasa.
- API: **http://localhost:3001/health** → `{"ok":true,"monsters":9}`
- Para detener: **Ctrl + C** en cada terminal.

Otros comandos útiles

Comando	Qué hace
`pnpm test`	Corre las 82 pruebas de todo el monorepo (con cobertura del engine).
`pnpm lint`	ESLint en todos los paquetes.
`pnpm typecheck`	Revisa los tipos de TypeScript en todo el repo.

`pnpm --filter @snac-hunters/infra synth`	Genera la plantilla de AWS (sin cuenta).
`pnpm --filter @snac-hunters/web build`	Build de producción en `apps/web/dist`.

Problemas comunes

Síntoma	Solución
Error `EADDRINUSE` (puerto ocupado)	Quedó un proceso anterior. En PowerShell: `Get-NetTCPConnection -LocalPort 3001,5173 | % { Stop-Process -Id $_.OwningProcess -Force }`
Página en blanco	Esperar unos segundos y recargar (**Ctrl + F5**). Revisar que la terminal de la web siga abierta.
Al terminar sale "⚠ API no disponible"	La terminal 1 (API) no está corriendo.
El celular no abre la página	Usar la dirección **Network**, no `localhost`; misma red Wi-Fi; permitir Node en el firewall de Windows (redes privadas).
Quiero empezar en ceros	Botón **"Reiniciar progreso"** en la pantalla de inicio.
`pnpm` no se reconoce	`npm install -g pnpm@10` y abrir una terminal nueva.

9. Guion de la demo (10 minutos)

Checklist antes de entrar al salón

- ☐ `pnpm install` ya hecho en la laptop que se proyecta.
- ☐ Terminal 1 (API) y terminal 2 (web) corriendo; `/health` responde.
- ☐ Celular abierto en la URL **Network** (para mostrar mobile-first).
- ☐ "Reiniciar progreso" pulsado.
- ☐ Abiertos en el editor: `combat.ts`, `config/game.json`, `snac-hunters-stack.ts`.
- ☐ Terminal 3 lista para `pnpm test` y `synth`.
- ☐ Decidir **quién presenta cada parte**: _____

Tiempo	Parte	Qué hacer y decir
0:00	**Arquitectura**	"El cliente pidió un tutorial para un jugador y un backend 100% AWS." Mostrar el diagrama de capas (sección 4). Idea clave: **el mismo engine corre en el navegador y en la Lambda**.
1:00	**Tutorial en vivo** (de preferencia en el celular)	1) "Aprender a jugar": el Huntmaster explica. 2) Tocar el dado verde: **se iluminan solo los espacios válidos**. "La UI no lo calcula; se lo pregunta al engine." 3) Colocar dados: el botón cambia entre Huir, Estudiar y Derrotar. 4) Seleccionar dados y Relanzar: aparece el **dado blanco**; el aviso dice si un corazón bloqueó el daño. 5) **Recargar la página a mitad del combate**: vuelve al mismo punto. 6) Resolver y elegir trofeo: **"✓ Validado por el servidor"**.
5:00	**Caza libre + analítica**	"Cazar otro SNAC": 2 opciones de tipos distintos. Si sale Porcupineapple, Picmimic o Porkchop, mostrar el espacio de corazón roto. Abrir el **botón DEV**: sesiones, % por cara, relanzamientos y tiempo del tutorial (meta < 5 min).
7:00	**Configuración**	En `config/game.json`, cambiar `"freeHunt": false` y guardar: el botón de caza desaparece **sin tocar código**. Regresarlo a `true`.
8:00	**Calidad y AWS**	`pnpm test`: 82 pruebas, 100% de líneas en el engine. `infra synth`: genera DynamoDB, Cognito, API Gateway, Lambda, S3 y CloudFront. Mostrar `snac-hunters-stack.ts`.
9:30	**Cierre**	Qué falta (sección 14) y siguiente sprint.

> **Plan B:** si algo falla en vivo (red, puerto, proyector), corre `pnpm test` y explica con este manual. Las pruebas demuestran la lógica aunque la UI no cargue.

10. Calidad: pruebas, cobertura y CI

Paquete	Pruebas	Qué cubren
`packages/engine`	**69**	Schemas, loaders, RNG y motor de combate. Cobertura: **100 % líneas, 100 % funciones, 98.7 % ramas** (mínimo exigido: 90 %).
`apps/api`	**9**	Integración del envío de puntuaciones: re-simulación, cliente tramposo, payload inválido, combate sin terminar, analítica y validez de los JSON reales de `config/`.
`apps/web`	**4**	Integración de persistencia: guardar y reanudar tras "cerrar el navegador", estado corrupto descartado y resumen de analítica.

Casos límite que pide el brief (todos con prueba)

- **Comodines:** cubren cualquier espacio de símbolo y "cualquiera", pero nunca un espacio de daño.
- **Dados blancos:** se agrega uno por relanzamiento; la cara en blanco queda en la reserva y se puede relanzar.
- **Daño y escudo simultáneos:** 2 caras de daño y 1 corazón en la misma tirada → se bloquea uno y se pierde 1 HPie.
- **KO y derrota en la misma tirada:** se conserva la derrota.
- **HPie nunca menor que 0 ni mayor que 6** (property-based, cientos de casos aleatorios).
- **Determinismo:** misma semilla y mismas acciones → mismo estado final (property-based).

Técnica de prueba: "tabla trucada"

Para probar casos raros sin depender de la suerte, las pruebas usan `fixedTable()`: una tabla de dados donde, por ejemplo, el dado blanco **siempre** saca daño. Así cada caso límite es reproducible.

Integración continua (GitHub Actions)

```
push / pull request ─▶ pnpm install --frozen-lockfile ─▶ pnpm lint ─▶ pnpm typecheck ─▶ pnpm test
```

Si cualquier paso falla, el CI marca rojo y no se debe hacer merge.

11. Decisiones de diseño

Donde el manual del juego no era claro, tomamos una decisión razonable y la **registramos** en `docs/decisiones.md`. El brief del cliente lo pide así.

ID	Tema	Decisión
D-01	Generador aleatorio	Mulberry32 con semilla: determinista, sin dependencias, unas 10 líneas.
D-02	Validar 4 espacios	`z.tuple` de 4: longitud exacta y tipo preciso.
D-03	Caras del dado	Confirmado con el Dice-guide: `heart`, `wild` y `damage` son caras distintas.
D-04	Loader puro	`loadMonsters(raw: unknown)` no lee archivos; el engine no hace I/O.
D-05	Vida máxima	6 HPie, en `config/game.json`.
D-06	Datos del cliente	Los 9 monstruos reales del Excel; sin HP para monstruos normales; colores de trofeo traducidos a símbolos.
D-07	Espacio de corazón roto	Una cara de daño lo cubre automáticamente, y el daño se paga con un corazón o con 1 HPie. **Pendiente de validar con el cliente.**
D-08	Escudo automático	Si hay un corazón en la reserva, bloquea el daño solo (los corazones no sirven para otra cosa).
D-09	Umbrales	4 = derrotar, 2–3 = estudiar, 1 = huir; con 0 no se puede resolver.
D-10	KO + derrota	Se conserva la derrota; se aplican el punto y la penalización de KO.
D-11	Validación	El servidor re-simula semilla + acciones. Riesgo conocido: hoy la semilla la genera el cliente.
D-12	Runtime Lambda	Node.js 22, porque Node 20 quedó deprecado en Lambda (abril de 2026).
D-13	Relanzar el dado blanco	Si muestra blanco o comodín y sigue en la reserva, se puede relanzar (FAQ del manual).

12. Preguntas probables

Pregunta	Respuesta corta
¿Por qué un monorepo?	Front, back, motor e infraestructura comparten tipos y código. Un cambio en las reglas se refleja en web y API a la vez, con un solo CI.
¿Por qué el engine es un paquete aparte?	Para que el navegador y la Lambda ejecuten **exactamente** las mismas reglas. Es TS puro y ESLint impide meterle React, AWS o archivos.
¿Cómo evitan que el jugador haga trampa?	El cliente solo manda semilla + acciones. La Lambda re-simula con `replayCombat`, usa los dados y la vida de la configuración del servidor y calcula el resultado. Una acción ilegal devuelve 422.
¿Qué es una semilla y por qué la usan?	Es el número inicial del generador aleatorio. Misma semilla → misma secuencia de dados. Permite repetir y verificar partidas.
¿Qué pasa si se cierra el navegador?	Zustand persiste en IndexedDB la semilla, las acciones y el progreso. Al volver se re-simula el combate. Si los datos están corruptos se descartan y el progreso se conserva.
¿Por qué IndexedDB y no localStorage?	Es asíncrona (no bloquea la UI), soporta más datos y es la recomendada para apps offline.
¿Para qué Zustand y no Redux?	Mucho menos código para el mismo resultado, con persistencia integrada. Suficiente para el tamaño del proyecto.
¿Dónde se guardan los datos en AWS?	En DynamoDB (single-table): `RUN#usuario` para partidas validadas y `EVENT#sesión` para analítica.
¿Cómo se autentica el jugador?	Cognito User Pool. API Gateway valida el token con un authorizer antes de llamar a la Lambda. En la web, Amplify obtiene el token. En local se juega como invitado.
¿Qué es CDK y por qué usarlo?	Infraestructura como código en TypeScript: es versionable, revisable y reproducible (dev y prod idénticos). Genera CloudFormation.
¿Por qué serverless?	Pago por uso, escala solo, sin servidores que administrar. Además, el cliente pidió solo AWS.
¿Cómo cambia el cliente la economía?	Editando `config/*.json`. Zod valida los valores. Siguiente paso: servirlos con AWS AppConfig para cambiarlos sin redeploy.
¿Qué es un feature toggle?	Un interruptor en `config/game.json` que activa o desactiva una función (tutorial, caza libre, pueblo…) sin cambiar código.
¿Cómo miden que el tutorial dura menos de 5 minutos?	La analítica registra `tutorial_start` y `tutorial_complete`; el botón DEV muestra el promedio en segundos.
¿Qué decidieron donde el manual era ambiguo?	Están en `docs/decisiones.md` (D-06 a D-13). Ejemplo: cómo se cubre el espacio de corazón roto (D-07).
¿Qué es property-based testing?	En lugar de unos cuantos ejemplos, fast-check genera cientos de entradas aleatorias y verifica que una propiedad siempre se cumpla.

¿Está desplegado?	No todavía. La infraestructura sintetiza sin errores; falta configurar una cuenta de AWS y correr `cdk deploy`.

13. Glosario

Término	Significado
Vertical slice	Una porción pequeña pero completa del producto que atraviesa todas las capas (UI, API, datos, infraestructura).
Monorepo	Un repositorio con varios paquetes relacionados.
Engine / motor	El paquete con las reglas del juego (`packages/engine`).
Semilla (seed)	Valor inicial de un generador aleatorio determinista.
Re-simulación (replay)	Volver a ejecutar una partida a partir de su semilla y acciones para verificar el resultado.
Serverless	Modelo donde el proveedor (AWS) administra los servidores; tú solo subes funciones y pagas por uso.
IaC (Infraestructura como código)	Definir los recursos de la nube en archivos de código (aquí, CDK).
CloudFormation	Servicio de AWS que crea recursos a partir de una plantilla; CDK la genera.
Authorizer	Componente de API Gateway que valida el token del usuario antes de dejar pasar la petición.
JWT	Token firmado que identifica al usuario; lo emite Cognito.
Single-table design	Patrón de DynamoDB donde distintos tipos de datos comparten una tabla y se distinguen por su clave.
OAC	Origin Access Control: permite que solo CloudFront lea el bucket S3 privado.
Feature toggle	Interruptor de configuración para activar o desactivar funciones sin cambiar código.
Zod	Librería que valida que los datos tengan la forma esperada en tiempo de ejecución.
IndexedDB	Base de datos integrada en el navegador; persiste aunque se cierre.
CI	Integración continua: GitHub Actions corre lint, tipos y pruebas en cada push.
HPie	La vida del cazador en el juego (máximo 6).
Push your luck	Mecánica de arriesgar más (relanzar) a cambio de un posible castigo (dado de daño).

14. Estado actual y siguientes pasos

Lo que ya funciona

- Tutorial interactivo completo (8 pasos) y caza libre con los 9 SNACs reales y su arte.
- Motor de reglas compartido, determinista, con 100 % de cobertura de líneas.
- Web ↔ API local de punta a punta, con validación por re-simulación.
- Persistencia en IndexedDB, incluida la reanudación de sesiones interrumpidas.
- Analítica local y en servidor, con botón DEV.
- Feature toggles y economía en `config/` .
- Infraestructura AWS completa en CDK; `cdk synth` sin errores ni advertencias.
- Lint, typecheck y 82 pruebas en verde.

Lo que falta (siguientes sprints)

- **Desplegar en AWS** (`cdk deploy`): requiere una cuenta y credenciales configuradas.
- **Pantalla de login** con Cognito (el cliente de Amplify ya está conectado).
- **Fase de pueblo** (rest, research, specialize, sparring, scavenge, misiones) y **jefe Pieous**; hoy están apagados por toggle.
- **Semilla emitida por el servidor** (`POST /runs/start`) para cerrar la última vía de trampa.
- **AWS AppConfig** para cambiar configuración y toggles sin redeploy.
- Analítica a escala: Firehose → S3 → Athena (roadmap del documento técnico).
- Validar con el cliente las decisiones pendientes (D-07).
- Pruebas E2E con Playwright.